

Code-Layout, Readability and Reusability

Date	14 July 2026
TeamID	SWUID2026221635
ProjectName	A Comprehensive Measure of Well-Being (HDI Prediction & Analysis)
MaximumMarks	5Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the code base in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/ Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Followed strict PEP 8 guidelines for Python and standard indentation for HTML/CSS
2	Proper File Structure	Files and folders are logically organized	Yes	Structured cleanly with templates/, static/, models/, and main app.py at the root.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables like <code>gni_per_capita</code> and <code>life_expectancy</code> are used instead of short abbreviations.
4	Function / Method Names	Functions are descriptively named	Yes	Clear names used such as <code>predict_hdi_tier()</code> and <code>load_ml_model()</code> .
5	Code Comments	Inline and block comments explain logic	Yes	Added docstrings to functions and brief inline comments for data transformations.
6	Modular Design	Code is split into reusable functions/modules	Yes	Machine Learning logic, route handling, and plot rendering are separated into modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Refactored redundant prediction blocks into a helper class to avoid duplication.

8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks handle database failures and invalid form input ranges securely.
---	----------------	--	-----	--

ReusableComponents/Modules:

S.No	Component / Module Name	Language /Technology	Where used	Reusability Level (High/Medium/Low)
1	ML Model Loader Pipeline (model_loader.py)	Python / Scikit-learn	Reused in backend routes to run test predictions on new user data.	High
2	Dynamic Graph Renderer (chart_generator.py)	Python / Matplotlib & Seaborn	Reused across prediction outputs and comparative dashboard tabs.	High
3	Form Input Sanitizer (validator.py)	JavaScript	Used across register, login, and slider submission pages.	Medium
4	Card Component Layout (card.html)	HTML / Tailwind CSS	Reused to display multiple country statistics side-by-side.	High
5				

OverallCodeQualityAssessment:

Aspect	Rating (1-5)	Comments
Code Layout &Structure	5	The file structure is clean and separates configuration from logical layers perfectly.
Readability	4	High readability with descriptive names and helpful function headers.
Reusability	5	The prediction pipeline is decoupled from Flask, allowing easy reuse in command-line tests.
Documentation / Comments	4	Well-placed comments explain mathematical constraints and prediction routing.
Overall Score	4.5	Highly structured, clean codebase that meets professional industry standards.